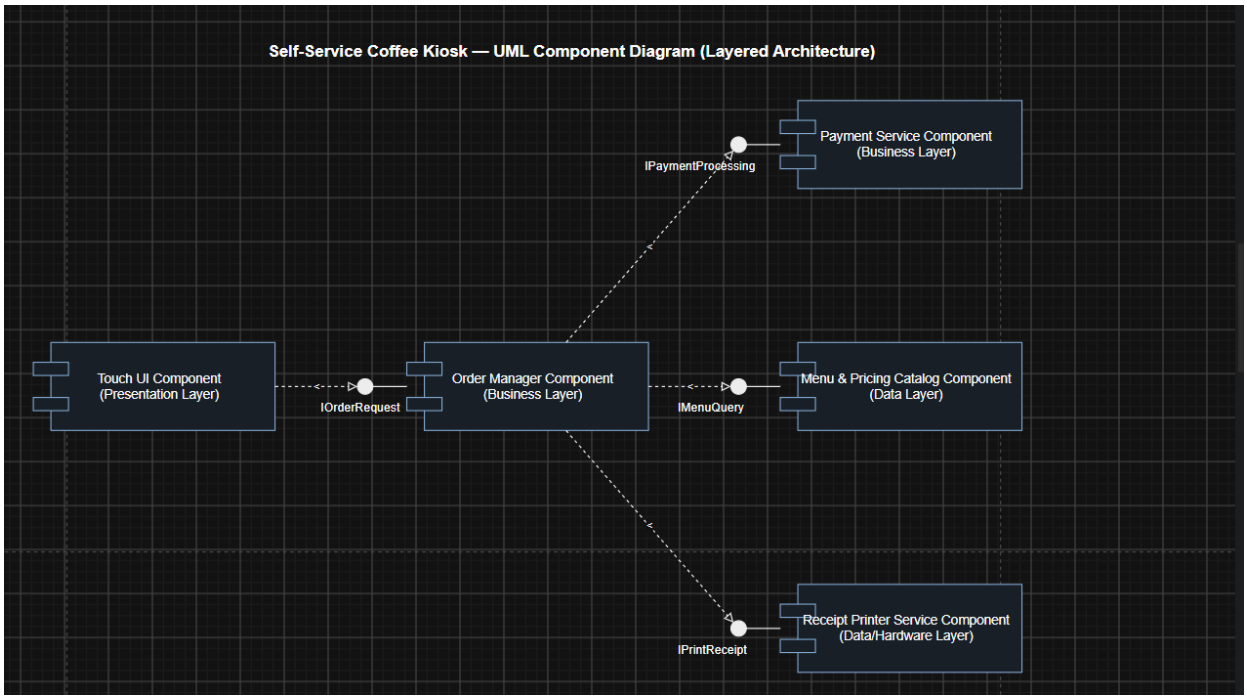
Lab 3 — Component Modelling & Architectural Pattern Selection

Name: Roopesh
SRN: PES1UG24AM393

1. Component Diagram



Component diagram for the Self-Service Coffee Kiosk System, drawn using a Layered Architecture. It shows the 5 components spread across the Presentation, Business, and Data/Hardware layers, along with the 4 interfaces connecting them (IOrderRequest, IPaymentProcessing, IMenuQuery, IPrintReceipt).

2. Written Justification

For the Self-Service Coffee Kiosk System, we went with a Layered Architecture.

Architectural Choice

The system is split into three layers. At the top is the Presentation layer, which is just the Touch UI Component - it handles the customer tapping through coffee type, size, and starting the payment. Below that is the Business layer, made up of the Order Manager and Payment Service, which is where the actual order logic and payment handling happen. At the bottom is the Data layer, covering the Menu & Pricing Catalog and the Receipt Printer Service, which deal with stored data and the hardware side of things. Each layer only talks to the one directly below it, through a fixed interface, so nothing skips over a layer.

Reason 1 — It's Just One Machine, Not a Distributed System

A coffee kiosk is a single physical terminal with a pretty small, fixed job: 3 coffee types, 2 sizes, card-only payments, one printer. There's no real need to scale different parts of it independently or worry about one service going down while another keeps running, which is basically the whole point of Microservices. Going that route here would just mean extra complexity - services talking to each other over a network, consistency issues - for a problem that doesn't actually exist on a single kiosk. A layered setup lets us build and update the whole thing as one unit, which fits how a kiosk actually gets deployed.

Reason 2 — Keeps the Touchscreen, Logic, and Hardware From Getting Tangled

The kiosk has to juggle a touchscreen, a card reader, and a physical printer at the same time. By keeping things layered, the hardware and data-facing code (Data layer) stays separate from the UI code (Presentation layer) and the order/payment rules (Business layer). So if we need to tweak the touchscreen UI or swap the printer driver later, we're not risking breaking the

payment or order logic in the process. It also happens to line up nicely with how the lab wants the components grouped - Order Manager and Payment Service as given, plus the three we added.

Security Advantage

Only two components ever touch sensitive data: the Menu & Pricing Catalog (pricing info) and the Payment Service (transaction details). The Touch UI never talks to the database or the payment processor directly - every request from it has to go through the Order Manager's IOrderRequest interface first. So even if something in the UI layer gets compromised, it can't reach pricing data or the payment channel directly, since everything is gated behind the Business layer.

Performance Benefit

Since the Menu & Pricing Catalog sits behind a clean IMenuQuery interface in its own layer, the Order Manager can just cache the price list (it's only 3 drinks × 2 sizes, so it barely changes) instead of hitting the data layer every single time someone taps an item. That keeps the ordering flow snappy on the customer side, which matters a lot for a self-service kiosk - nobody wants to stand around waiting at checkout.